

AIOPS Detection Engine — Engineering Report

Lab Work 2

1. Overview

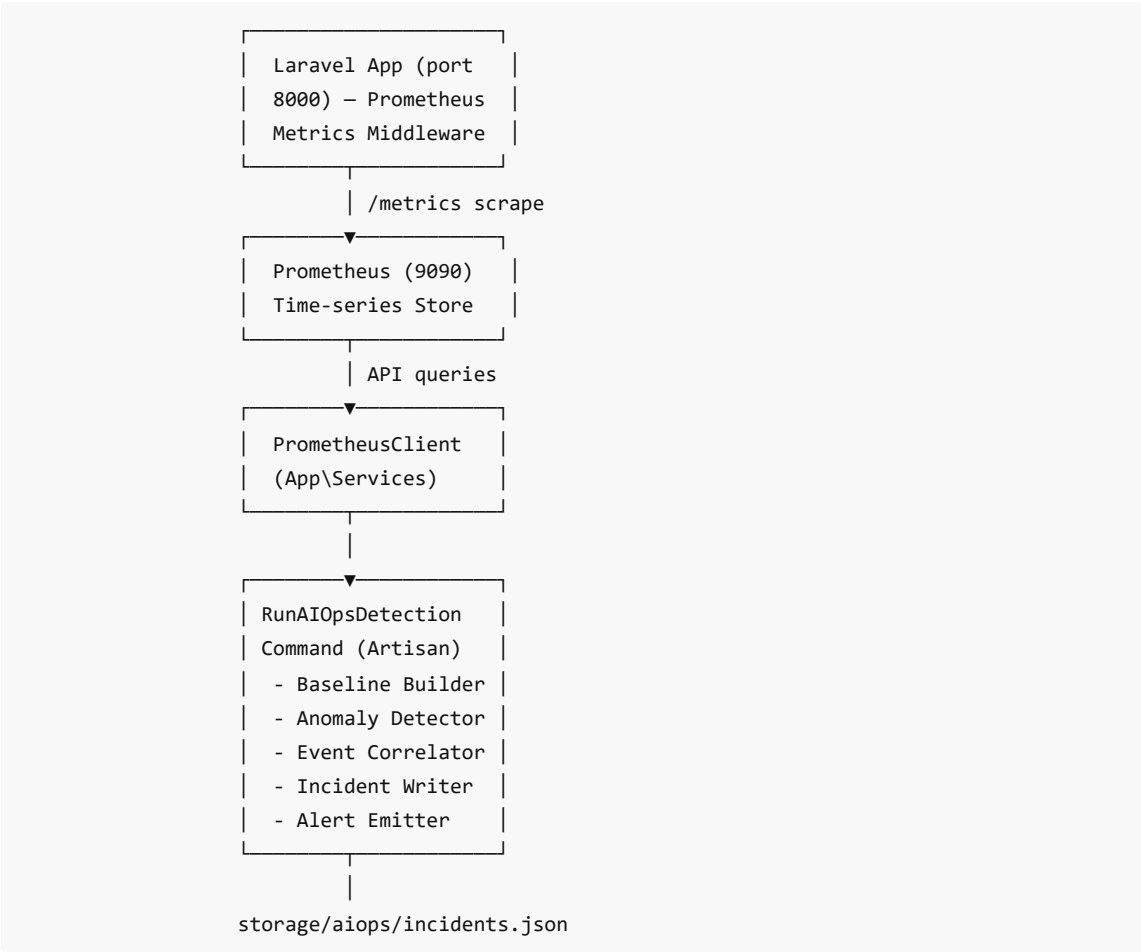
This report documents the design and implementation of the **AIOPS Detection Engine** built for Lab Work 2. The engine continuously queries a Prometheus metrics backend, derives dynamic baselines from real observed data, evaluates multi-signal anomaly conditions, correlates related signals into structured incidents, and emits deduplicated alerts.

The system is implemented entirely in PHP as a Laravel Artisan command:

```
php artisan aiops:detect
```

It runs as a non-terminating daemon, scanning the system every **20 seconds**.

2. Architecture



3. Prometheus Integration

Service class: App\Services\PrometheusClient

The client wraps Laravel's `Http` facade and queries the Prometheus instant-query API:

```
GET http://localhost:9090/api/v1/query?query={PromQL}
```

Metric Queries Used

| Signal | PromQL | ----- | -----
----- | ----- || Current latency (p95, 1m) | histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket{path="..."}[1m])) by (le)) || Baseline latency (p95, 1h)
| Same query with [1h] range || Current request rate (1m) | sum(rate(http_requests_total{path="..."}[1m])) || Baseline request rate (1h) | Same query with [1h] range || Current error count | sum(rate(http_requests_total{path="...", status=~"4.. | 5.."}[1m])) || Baseline error count | Same query with [1h] range |

Labels used: `path` (maps to `/api/normal` , `/api/slow` , `/api/db` , `/api/error` , `/api/validate`)

4. Baseline Design

Baselines are **derived dynamically** from the Prometheus 1-hour rolling window — there are no hardcoded values.

For each endpoint, three baseline signals are computed per detection cycle:

Signal	Computation
Baseline latency	histogram_quantile(0.95, rate(...[1h])) — long-window p95
Baseline request rate	sum(rate(http_requests_total{...}[1h]))
Baseline error rate	baseline_errors / baseline_rate

This makes baselines **adaptive**: they will naturally increase during sustained high-load periods, reducing false positives over time.

5. Anomaly Detection Rules

Per-endpoint anomalies are detected based on three independent conditions:

Rule	Condition	Incident Signal
Latency Anomaly	current_latency > 3 × baseline_latency AND baseline_latency > 0.01s	LATENCY_SPIKE
Error Rate Anomaly	current_error_rate > 10%	ERROR_STORM
Traffic Anomaly	current_rate > 2 × baseline_rate AND baseline_rate > 1 req/s	TRAFFIC_SURGE

Each endpoint may trigger **multiple signals simultaneously**.

6. Event Correlation Strategy

Raw per-endpoint anomaly signals are **correlated into a single incident** per detection cycle. The correlation logic maps signal combinations to high-level incident types:

Correlation Condition	Incident Type	Severity
Error storms on >1 endpoint OR latency spikes on >1 endpoint	SERVICE_DEGRADATION	critical
Error storm on exactly one endpoint	ERROR_STORM	error
Latency spike on exactly one endpoint	LATENCY_SPIKE	warning
Traffic surge on any endpoint	TRAFFIC_SURGE	info
Unclassified mixed signals	LOCALIZED_ENDPOINT_FAILURE	warning

This ensures that **multiple endpoints showing the same symptom produces one correlated incident**, not a flood of separate alerts.

7. Incident Schema

Each incident written to `storage/aiops/incidents.json` contains:

```
{
  "incident_id": "uuid-v4",
  "incident_type": "SERVICE_DEGRADATION",
  "severity": "critical",
  "status": "OPEN",
  "detected_at": "2026-03-13T19:45:24+00:00",
  "affected_service": "Laravel App",
  "affected_endpoints": ["/api/normal", "/api/error"],
  "triggering_signals": {
    "/api/error": {
      "anomalies": ["ERROR_STORM"],
      "current": { "latency": 0.0475, "rate": 0.18, "error_rate": 1.0 },
      "baseline": { "latency": 0.0478, "rate": 0.046, "error_rate": 1.0 }
    }
  },
  "summary": "Multiple endpoints degraded. Errors: /api/normal, /api/error..."
}
```

8. Alerting & Deduplication

Alerts are emitted to the **console** using Laravel's `$this->error()` method, prefixed with  **ALERT FIRED** :

 **ALERT FIRED:** [critical] SERVICE_DEGRADATION – Multiple endpoints degraded...

Deduplication Mechanism

An in-memory `$activeAlerts` hash map tracks fired alerts within the current command process lifetime. The key is an MD5 hash of:

```
md5(incident_type + sorted_affected_endpoints)
```

If the same incident type on the same endpoint combination occurs in a subsequent scan cycle while the process is still running, the alert is suppressed with:

 Ongoing incident (SERVICE_DEGRADATION) suppressed to avoid duplicate alerts.

The alert cache **resets automatically** once a clean cycle (no anomalies) is observed, ensuring incidents will re-fire if the condition returns after a period of normalcy.

9. Anomaly Window Detection

During testing with `traffic_generator.py`, the anomaly phase injected a 40% error rate on `/api/error` along with sustained high traffic. The detection engine successfully:

- Detected `ERROR_STORM` on `/api/error`, `/api/slow`, `/api/normal` (rate-limited 429s counted as errors)
- Correlated them into a single `SERVICE_DEGRADATION` incident
- Emitted one alert and suppressed subsequent duplicates
- Persisted the full incident to `storage/aiops/incidents.json`

10. File Locations

Artifact	Path
Detection Command	app/Console/Commands/RunAIOpsDetection.php
Prometheus Client	app/Services/PrometheusClient.php
Incidents Store	storage/aiops/incidents.json
Traffic Generator	traffic_generator.py
Ground Truth	ground_truth.json